

Reproducing LoRA on RoBERTa-base

CS 4782 Final Project Summary

Lucas He, Partner

Cornell University, Spring 2026

1. Introduction

Fine-tuning a 175B-parameter language model for every downstream task is prohibitive: a fresh full copy of the model per task costs as much as the original training run and produces a 350 GB checkpoint. **LoRA: Low-Rank Adaptation of Large Language Models** (Hu et al., 2021)¹ sidesteps this by freezing the pre-trained weights and adding a trainable *rank- r* decomposition $\Delta W = BA$ in parallel to selected weight matrices. The forward pass becomes $h = W_0x + (\alpha/r)BAx$, with $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$. The paper’s central empirical claim is that this update has very low intrinsic rank — even $r=1$ or $r=2$ is enough to match full fine-tuning on many tasks.

2. Chosen result

We reproduce a slice of Table 2 of Hu et al. (2021): **RoBERTa-base on GLUE**, comparing LoRA against full fine-tuning. Specifically we target the **MRPC** cell (RoBERTa_{base} + LoRA: 87.5; FT: 90.2) as the headline result, and we additionally rerun the **rank ablation** of Table 6 ($r \in \{1, 2, 4, 8\}$) on RoBERTa-base / MRPC as our independent experiment. We chose this slice because (i) it tests the central claim of the paper at a scale a single Apple M-series GPU can handle in ~ 5 min per run, and (ii) it lets us evaluate *both* the parameter-efficiency claim (LoRA matches FT with a tiny fraction of the params) and the intrinsic-rank claim (rank doesn’t matter much) with the same backbone.

3. Methodology

Implementation. We wrote LoRA from scratch in ~ 60 lines (code/lora.py): a LoRALinear module wraps an nn.Linear, adds trainable lora_A (Kaiming-init) and lora_B (zero-init), and freezes the base linear. A helper walks the model and replaces every attention query and value projection with a LoRALinear of the chosen rank — following Table 5’s recommendation that $\{W_q, W_v\}$ is the best two-target configuration under a fixed parameter budget. We mark only LoRA parameters and the new classification head as trainable. We deliberately avoid peft so we know exactly what is being trained.

Datasets. GLUE MRPC (3.7k paraphrase pairs, accuracy) and SST-2 (67k sentiment sentences, accuracy), via datasets.load_dataset.

Training. AdamW (weight decay 0.01), linear warmup over 6% of steps then linear decay. LoRA at $lr=5 \times 10^{-4}$, full fine-tuning at $lr=2 \times 10^{-5}$. Batch size 32, max sequence length 128, seed 42. MRPC: 3 epochs (115 steps each); SST-2: 2000 steps (~ 1 epoch) due to wall-clock budget. Hardware: one Apple M-series GPU via PyTorch MPS.

Modifications. (a) RoBERTa-base instead of the paper’s headline GPT-3 175B (the paper still reports a RoBERTa-base cell, so we reproduce that). (b) We add a *head-only* baseline (frozen body, train only the classifier) to verify that LoRA’s lift is genuine. (c) The rank ablation is on RoBERTa-base / MRPC rather than GPT-3 / WikiSQL.

¹[arXiv:2106.09685](https://arxiv.org/abs/2106.09685)

4. Results & analysis

Headline (MRPC). Full FT 88.2%; LoRA $r=8$ 86.3%; LoRA $r=1$ 86.0%; head-only 68.4%. Our LoRA number is ~ 1.2 points below the paper’s reported RoBERTa_{base} + LoRA (87.5%) — plausibly because we use a single seed and fixed LR (the paper sweeps), and start from RoBERTa-base instead of the MNLI-warm-started checkpoint that some prior GLUE work uses for MRPC. The 18-point gap between LoRA and head-only is the load-bearing observation: LoRA’s lift is from the low-rank adaptation, not from bookkeeping in the new classifier head.

Rank ablation (MRPC).

r	1	2	4	8
Trainable params	629K	666K	739K	887K
MRPC accuracy	0.860	0.858	0.865	0.863

The spread across ranks is 0.7 points, well within seed-to-seed noise on the 408-example MRPC validation set (~ 0.24 points per misclassified example). This reproduces the qualitative finding of Table 6 of the paper: increasing r does not unlock new directions in the update, suggesting ΔW has a very low intrinsic rank.

Second task (SST-2). LoRA $r=8$ reaches 93.1% in a ~ 1 -epoch budget; paper reports 95.1% at the full sweep, a 2-point gap that we expect closes with more steps.

Discrepancies / challenges. (i) *The new classifier head dominates the trainable count* at 125M scale: at $r=8$, ~ 295 K params are LoRA and ~ 592 K are the head. The paper’s “10,000 $\times$ headline” is for GPT-3 175B where the head is negligible — we report both the 0.7% trainable share and the 140 $\times$ MRPC comparison rather than presenting the GPT-3 storage claim as our measured result. (ii) MRPC’s 408-example validation set has high seed-to-seed variance; with more compute we would average over 3+ seeds. (iii) Apple MPS throughput is $\sim 1/3$ – $1/4$ of a V100, which set the ceiling on what we could sweep.

In context. The reproduction supports the paper’s core argument: fine-tuning is overparametrized, and a tiny additive low-rank update suffices to adapt a pretrained Transformer. That this holds at 125M — not just at 175B — is encouraging for practitioners on modest hardware.

5. Reflections

- **From-scratch was the right call.** LoRA’s machinery is small enough that re-implementing it forced us to internalize what’s frozen, what’s trained, and how the update scales with r and α . Calling `peft.LoraConfig(...)` would have left that opaque.
- **The crucial hyperparameter wasn’t r — it was the LR.** LoRA needs roughly $25\times$ the LR of full fine-tuning ($5e-4$ vs. $2e-5$) because the matrices are tiny and zero-initialized; runs at $2e-5$ stayed near random.
- **Future directions.** (a) LoRA on MLP layers (the paper defers this); (b) the subspace-overlap test from §7.2 — do different seeds learn the same low-rank subspace? (c) full-rank fine-tuning on *just* W_q and W_v , to separate “low rank” from “few parameters.”
- **With more compute:** 3 seeds per cell, LR sweep per rank, and DeBERTa-XXL on one task — the paper’s DeBERTa numbers are its sharpest match to full FT and would make for a more striking headline.

6. References

Hu et al. (2021), *LoRA*, arXiv:2106.09685. Aghajanyan et al. (2020), *Intrinsic Dimensionality*. Liu et al. (2019), *RoBERTa*, arXiv:1907.11692. Wang et al. (2019), *GLUE*. Tools: PyTorch, HF Transformers/Datasets, Tectonic.